

Ministry of Higher Education
Kabul university
faculty of
Information & communication
technology

C++ Programming

Overview

- Basic Element of C++
- Identifiers
- Keywords
- Data types
- Variables
- Declaring and Initializing Variables
- Operators
- precedence
- Expressions
- Type Conversion (Casting)
- Cout, cin, Iostream

identifiers

An identifier is a user-defined name used to identify variables, functions, or other program elements in C++.

Reserved words

- Reserved words are also called keywords. The letters that make up a reserved word are always lowercase.
- Keywords are predefined words in C++ that the compiler understands and you cannot redefine.

Rules for Naming Identifiers in C++

1. Reserved words cannot be used as identifiers in a program.
2. In C++, identifiers are names of things.
3. A C++ identifier consists of letters, digits, and underscores and must begin with a letter or underscore.

Data type

A data type in C++ is a classification that specifies **the kind of data** a variable can store, **the amount of memory** allocated for it, and **the operations** that can be performed on that data.

Kind of data type in c++

- Simple data type
- Structured data type
- Pointers

Simple data type

The simple data type is the fundamental data type in C++ and store single, basic values and are used for simple calculations.

- There are three categories of simple data:
 1. Integral, which is a data type that deals with integers, or numbers without a decimal part
 2. Floating-point, which is a data type that deals with decimal numbers
 3. Enumeration, which is a user-defined data type

Data Type	Size (Bytes)	Example	Description
int	4byte	10, -25, 100	Standard integer number
short	2byte	1000	Small-range integer
long	8byte	1000000000	Large-range integer
float	4byte	3.14	Decimal number with single precision
double	8byte	3.14159	Decimal number with double precision
char	1byte	'A', 'b', '1'	Single character
bool	1byte	true, false	Boolean value (true or false)
void	0	—	No data, used in functions with no return

Structured data types are used to store multiple values (same or different types) under one name.

A pointer is a data type that stores the memory address of another variable.

variables

A variable in C++ is a named memory location that stores a value of a specific data type.

Declaring and Initializing Variables

Syntax for declaration a variable

datatype variable name;

Example

int x;

double y;

Syntax for declaration and initialization of variable

Datatype variable_name=value;

Example

int x=9;

Double y=5.4;

Key point

- A named constant is initialized when it is declared.
- All variables must be declared before they can be used.
- C++ does not automatically initialize variables.
- Every variable has a name, a value, a data type, and a size.
- When a new value is assigned to a variable, the old value is lost.

- **Named constant:** A memory location whose content is not allowed to change during program executions
- To allocate memory, we use C++'s declaration statements. The syntax to declare a named constant is.
- A named constant is declared by using the reserved word **const**.

const dataType identifier = value;

Example:

const double CONVERSION = 2.54;

const int NO_OF_STUDENTS = 20;

const char BLANK = ' ';

Operator

An operator in C++ is a symbol that instructs the compiler to perform a specific operation on one or more operands.

Kind of operator

- Arithmetic operator
- Logical operator
- Relational operator
- Assignment operator
- Increment and Decrement Operators

Arithmetic operator

Operator	Description	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 2	5
%	Modulus (remainder)	10 % 3	1

Increment and decrement operator

++	Increment	a++ or ++a	a = a+1
--	Decrement	a-- or --a	a = a-1

Relational operator

Operator	Description	Example	Result
==	Equal to	a == b	true/false
!=	Not equal to	a != b	true/false
>	Greater than	a > b	true/false
<	Less than	a < b	true/false
>=	Greater or equal	a >= b	true/false
<=	Less or equal	a <= b	true/false

Logical operator

Operator	Description	Example	Result
&&	Logical AND	a>0 && b>0	true if both true
	Logical OR		Logical OR
!	Logical NOT	!true	false

Not operator (!)

Expression	!(Expression)
true (nonzero)	false (0)
false (0)	true (1)

AND operator &&

Expression1	Expression2	Expression1 && Expression2
true (nonzero)	true (nonzero)	true (1)
true (nonzero)	false (0)	false (0)
false (0)	true (nonzero)	false (0)
false (0)	false (0)	false (0)

OR operator ||

Expression1	Expression2	Expression1 Expression2
true (nonzero)	true (nonzero)	true (1)
true (nonzero)	false (0)	true (1)
false (0)	true (nonzero)	true (1)
false (0)	false (0)	false (0)

Assignment operator

Operator	Description	Example
=	Simple assignment	a = 5
+=	Add and assign	a += 3 (a = a + 3)
-=	Subtract and assign	a -= 2
*=	Multiply and assign	a *= 2
/=	Divide and assign	a /= 2
%=	Modulus and assign	a %= 3

Order of precedence

Operator precedence in C++ defines the priority of operators in an expression. Operators with higher precedence are evaluated first, and operators with lower precedence are evaluated later.

Order of precedence

Operators	Precedence
!, +, - (unary operators)	first
*, /, %	second
+, -	third
<, <=, >=, >	fourth
==, !=	fifth
& &	sixth
	seventh
= (assignment operator)	last

Type conversion (casting)

Type conversion in C++ is the process of converting a value or variable from one data type to another.

There is two kind of type conversation in c++.

1. Implicit type conversion
2. Explicit type conversion

Implicit type conversion

Implicit type conversion is the conversion of one type of data to another type automatically by the compiler when needed.

Example

```
int inttype= 10;
```

```
double doubletype = intValue + 5.5;
```

```
#include <iostream>
using namespace std;

int main() {

    int a = 5;
    double b = 2.5;

    double result = a + b;
    cout << "Result: " << result << endl;
    return 0;
}
```

Output:

7.5

Explicit type conversion

Explicit type conversion, also called type casting is the conversion of one type of data to another type manually by a programmer.

static_cast<dataTypeName>(expression)

```
#include <iostream>
using namespace std;
```

```
int main() {
    double pi = 3.14;

    int a = (int)pi; //first style
    int b = static_cast<int>(pi); // second style
    cout << "a = " << a << endl; // Output: 3
    cout << "b = " << b << endl; // Output: 3

    return 0;
}
```

Output

3
3

Basic Input / Output in C++

- In C++, data is read and written using streams, which are sequences of bytes.
- **Input stream:** A sequence of characters from an input device to the computer.
- **Output stream:** A sequence of characters from the computer to an output device.
- These streams are defined in the **<iostream>** header file.
- The most common stream objects are - "**cin**": for taking input and "**cout**": for displaying output.

What is iostream in c++

In C++, the <iostream> header file is part of the standard library and provides the core functionality for standard input and output operations using the concept of streams.

The name "iostream" is an abbreviation for **Input/Output Stream**.

Concept of a Stream

- A stream in C++ is an infinite sequence of characters from a source to a destination.
- **Input stream:** A sequence of characters from an input device to the computer.
- **Output stream:** A sequence of characters from the computer to an output device.

#include <headerFileName>

Example

#include <iostream>

cout is used to display output on the screen and It represents the **standard output stream**.

cin is used to read input from the keyboard and It represents the **standard input stream**.

- **istream cin;**
- **ostream cout;**

- In C++, `>>` is called the stream extraction operator.
- Input from the standard input device is accomplished by using `cin` and the stream extraction operator `>>`.
- In C++, `<<` is called the stream insertion operator.
- Output of the program to the standard output device is accomplished by using `cout` and the stream insertion operator `<<`.
- The manipulator `endl` positions the insertion point at the beginning of the next line on an output device.

Namespace is like a special container or a label that holds a group of names - like variables, functions, or classes etc...

std is the standard namespace in C++ that contains all components of the C++ Standard Library, including input/output objects, containers, algorithms, and utility functions.

A namespace is like a box that groups names, and std is the name of the standard namespace in C++

Examples 1:

```
int number;  
cin >> number;  
cout << number << endl;
```

Example 2:

```
int a;  
double b;  
  
cin >> a>> b;  
cout <<a<< endl;  
cout <<b<< endl;
```

cin and the **get** Function

- The function **get()** is used to read data on a character-by-character basis and does not skip any whitespace characters.

Syntax

cin.get(varChar);

getline() is a C++ function that reads an entire line of text from the input stream, including spaces, until the newline character (Enter) is encountered.

getline(istreamVar, strVar);

Example:

```
#include <iostream>
using namespace std;

int main() {
    char ch;
    string line;

    cout << "Enter a character: ";
    cin.get(ch); cout << "You entered: " << ch <<
endl;

    cout << "Enter a line: ";
    getline(cin, line);
    cout << "You entered: " << line << endl;

    return 0;
}
```


- All C++ statements must end with a semicolon. The semicolon is also called a statement terminator.

Escape Sequence		Description
<code>\n</code>	Newline	Cursor moves to the beginning of the next line
<code>\t</code>	Tab Cursor	moves to the next tab stop
<code>\b</code>	Backspace	Cursor moves one space to the left
<code>\r</code>	Return	Cursor moves to the beginning of the current line (not the next line
<code>\\</code>	Backslash	Backslash is printed
<code>\'</code>	Single quotation	Single quotation mark is printed
<code>\"</code>	Double quotation	Double quotation mark is printed

comments

A **comment** in C++ is a **piece of text included in the source code** that is **ignored by the compiler**.

Type of comment

1. **A single-line comment** starts with the pair of symbols `//` anywhere in the line.
2. **Multiline comments** are enclosed between `/*` and `*/`.